

SprintCraft: A Context Aware User Story Generator F24-012-D-SprintCraft

Project Team

Muhammad Shamil Umar	21I-1786
Hamiz Ahmed Siddiqui	21I-1678
Afnaan Asif	21I-1387

Session 2021-2025

Supervised by

Mr. Pir Sami Ullah Shah



Department of Data Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

September, 2024

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	User Input Module	2
1.3.2	Natural Language Processing (NLP) Engine	2
1.3.3	User Story Generation Module	3
1.3.4	Story Review and Editing Module	3
1.3.5	Version Control and Tracking Module	4
1.3.6	Security and Authentication Module	4
1.4	User Classes and Characteristics	5
2	Project Requirements	7
2.1	Use-case Diagrams	7
2.1.1	User Input Module	7
2.1.2	User Story Generation Module	11
2.1.3	Version Control and Tracking Module	16
2.1.4	Security and Authentication Module	20
2.2	Functional Requirements	23
2.2.1	User Input	23
2.2.2	Natural Language Processing (NLP) Engine	24
2.2.3	User Story Generation	25
2.2.4	Story Review and Editing	26
2.2.5	Version Control and Tracking	27
2.2.6	Security and Authentication	27
2.3	Non-Functional Requirements	28
2.3.1	Performance	28
2.3.2	Scalability	28
2.3.3	Security	29
2.3.4	Usability	29
2.3.5	Reliability	29
2.3.6	Maintainability	29

2.4	Domain Model	30
3	System Overview	33
3.1	Architectural Design	33
3.1.1	Box and Line Diagram	33
3.1.2	Model, View and Controller (MVC) Diagram	35
3.2	Design Models	36
3.2.1	Module 1: User Input Module	36
3.2.2	Module 2: Natural Language Processing (NLP) Engine	38
3.2.3	Module 3: User Story Generation Module	40
3.2.4	Module 4: Story Review and Editing Module	42
3.2.5	Module 5: Version Control and Tracking Module	44
3.2.6	Module 6: Security and Authentication Module	46
3.3	Data Design	48

List of Figures

2.1	Use Case diagram for User Input Module	7
2.2	Use Case diagram for User Story Generation Module	11
2.3	Use Case diagram for Version Control and Tracking Module	16
2.4	Use Case diagram for Security and Authentication Module	20
2.5	Domain Model Diagram	30
3.1	Box and Line diagram for complete architecture	33
3.2	MVC diagram for complete architecture	35
3.3	Module 1: Activity Diagram	36
3.4	Module 1: System Sequence Diagram	37
3.5	Module 2: Activity Diagram	38
3.6	Module 2: System Sequence Diagram	39
3.7	Module 3: Activity Diagram	40
3.8	Module 3: System Sequence Diagram	41
3.9	Module 4: Activity Diagram	42
3.10	Module 4: System Sequence Diagram	43
3.11	Module 5: Activity Diagram	44
3.12	Module 5: System Sequence Diagram	45
3.13	Module 6: Activity Diagram	46
3.14	Module 6: System Sequence Diagram	47
3.15	Schema Design for complete architecture	48

List of Tables

2.1	Use Case Descriptions	8
2.2	Detail Use Case for Module 1, Use Case 1	9
2.3	Detail Use Case for Module 1, Use Case 2	9
2.4	Detail Use Case for Module 1, Use Case 3	10
2.5	Detail Use Case for Module 1, Use Case 4	10
2.6	Use Case Descriptions for Module 2	12
2.7	Detail Use Case for Module 2, Use Case 1	12
2.8	Detail Use Case for Module 2, Use Case 2	13
2.9	Detail Use Case for Module 2, Use Case 3	13
2.10	Detail Use Case for Module 2, Use Case 4	14
2.11	Detail Use Case for Module 2, Use Case 5	14
2.12	Detail Use Case for Module 2, Use Case 6	15
2.13	Use Case Descriptions for Module 5	17
2.14	Detail Use Case for Module 5, Use Case 1	17
2.15	Detail Use Case for Module 5, Use Case 2	18
2.16	Detail Use Case for Module 5, Use Case 3	18
2.17	Detail Use Case for Module 5, Use Case 4	19
2.18	Use Case Descriptions for Module 6	21
2.19	Detail Use Case for Module 6, Use Case 1	21
2.20	Detail Use Case for Module 6, Use Case 2	22
2.21	Detail Use Case for Module 6, Use Case 3	22
2.22	Detail Use Case for Module 6, Use Case 4	22
2.23	Detail Use Case for Module 6, Use Case 5	23

Chapter 1

Introduction

This document outlines development of a system that will be designed to automate the generation of user stories from Software Requirement Specification (SRS) document, along with prompt specifying desired feature that needs to be implemented. The system aims to streamline the process of converting client's requirements into actionable and understandable user stories for developers which is core component in agile development life cycle. The system not only focuses on user stories generation but also evaluates them using pre-defined thresholds. The thresholds are based on scoring the user story based on its completeness, clarity and feasibility. The evaluation stage ensures that each user story contains all components, is easily understandable and is feasible for developers. The system is being created in response to increasing demand for software development processes to be automated, especially when it comes to breaking down and translating raw requirements into manageable tasks for development teams.

1.1 Problem Statement

In software development life cycle, especially in agile development, converting clients' requirements into actionable and manageable user stories is a time-consuming process and error prone. When creating user stories manually, there is substantial risk of inconsistencies, incomplete coverage of requirements and probable miscommunication among stakeholders and development teams. This project aims to automate the translation of SRS documents into comprehensive and consistent user stories which are aligned to specified requirements. Moreover, the system will include an evaluation stage that will make sure that each user story meets necessary criteria for quality and completeness. Each generated story will be checked against those criteria and scored based on 4 major categories: Completeness, Clarity, Feasibility and Value. The acceptable user story is one that is check all the mentioned criteria ensuring that each story is clearly describing the problem for

teams.

1.2 Scope

The project aims to create an intelligent software solution that automates the generation of user stories from Software Requirements Specification (SRS) documents. By accepting an SRS document and a user prompt as inputs, the software will generate contextually accurate user stories. Key functionalities include parsing and analyzing the SRS, maintaining context throughout the process, ensuring alignment with the specified requirements and self evaluating the user stories. The software will allow customization through prompts and will incorporate self-evaluation mechanisms to ensure accurate and relevant user story generation. Additionally, the tool will maintain a history of interactions for consistent output across sessions. The scope of the project includes developing a user-friendly interface and a backend engine consisting of fine tuned LLMs working in harmony.

1.3 Modules

1.3.1 User Input Module

This module allows users (e.g., Product Managers, Business Analysts) to upload an SRS document along with prompts. It provides a user-friendly interface where users can specify what kind of user stories they want to generate, whether it's for functional, non-functional, or specific sections of the provided SRS.

Key Features:

1. SRS document upload
2. Prompt field for user story specification
3. Input validation for ensuring correct file format and prompt structure

1.3.2 Natural Language Processing (NLP) Engine

The NLP engine processes the uploaded SRS document and interprets the user's provided prompt. It extracts key elements such as requirements, actors, and system behaviors from the SRS and translates them into user stories. The engine ensures that technical language

is transformed into user-friendly language suitable for user stories.

Key Features:

1. Requirement extraction
2. Contextual understanding of the SRS document
3. Prompt-driven interpretation for tailored user stories

1.3.3 User Story Generation Module

This is the core module responsible for transforming the extracted data from the NLP engine into coherent user stories. It adheres to a predefined format (e.g., “As a [role], I want [goal], so that [benefit]”). The module ensures that both functional and non-functional requirements are covered, based on the input provided.

Key Features:

1. Auto-generation of user stories
2. Handling functional and non-functional requirements
3. Customization based on the user’s prompt

1.3.4 Story Review and Editing Module

After generating user stories, this module allows users (e.g., Product Managers, Business Analysts, Developers etc.) to review and edit the output. It provides an interface where users can modify the generated stories, add acceptance criteria, and provide feedback on any necessary changes or introduce a perspective for context.

Key Features:

1. User story editing
2. Acceptance criteria addition
3. Feedback mechanisms for adjusting story generation rules

1.3.5 Version Control and Tracking Module

This module tracks changes made to user stories, allowing users to maintain different versions. It ensures that the history of changes is available for future reference, aiding collaboration among team members.

Key Features:

1. Version history tracking
2. Story comparison between versions
3. Collaboration support with multiple users

1.3.6 Security and Authentication Module

This module handles user authentication and security, ensuring that only authorized personnel can access the system. It includes user login, password management, and session control features.

Key Features:

1. User authentication and login
2. Role-based access control
3. Session management and encryption

User class	Description
Product Manager	A Product Manager is responsible for overseeing product development. They will use the system to input an SRS document and prompt for generating user stories. Product Managers require detailed and accurate user stories that are aligned with business and technical requirements.
Software Developer	Software Developers use the generated user stories to implement features and functionality. Developers rely on clear, technical user stories with well-defined acceptance criteria and may need the ability to request revisions or more detailed descriptions.
Business Analyst	Business Analysts bridge the gap between stakeholders and the development team. Analysts refine the SRS before submission and use the system to ensure user stories align with business goals. They may work collaboratively to edit or expand stories as necessary.
QA Tester	QA Testers use the user stories to create test cases and scenarios. A QA Testers primary interaction with the system is to ensure stories have clearly defined acceptance criteria and are testable.
Stakeholders	Stakeholders review the output stories at a high level, ensuring they align with the project's vision. Their interaction is primarily for feedback and approval, as they may not be involved in the detailed creation or technical aspects of the stories.
Project Supervisor	Project Supervisors monitor overall progress and ensure that user stories align with defined goals. They check stories for consistency with the project scope and may use the system to generate reports or summaries based on the stories.

1.4 User Classes and Characteristics

The table 1.4 above outlines the various user classes anticipated to interact with the system that generates user stories from SRS documents. Each class is defined based on its role in the product development lifecycle, detailing their interactions with the system and the specific requirements they have from the generated user stories (Product managers, Software developers ,etc). Each class has distinct needs, ranging from high-level oversight to technical implementation. This classification ensures that the system caters to all user roles effectively, aligning the output with both business and technical objectives.

Chapter 2

Project Requirements

2.1 Use-case Diagrams

2.1.1 User Input Module

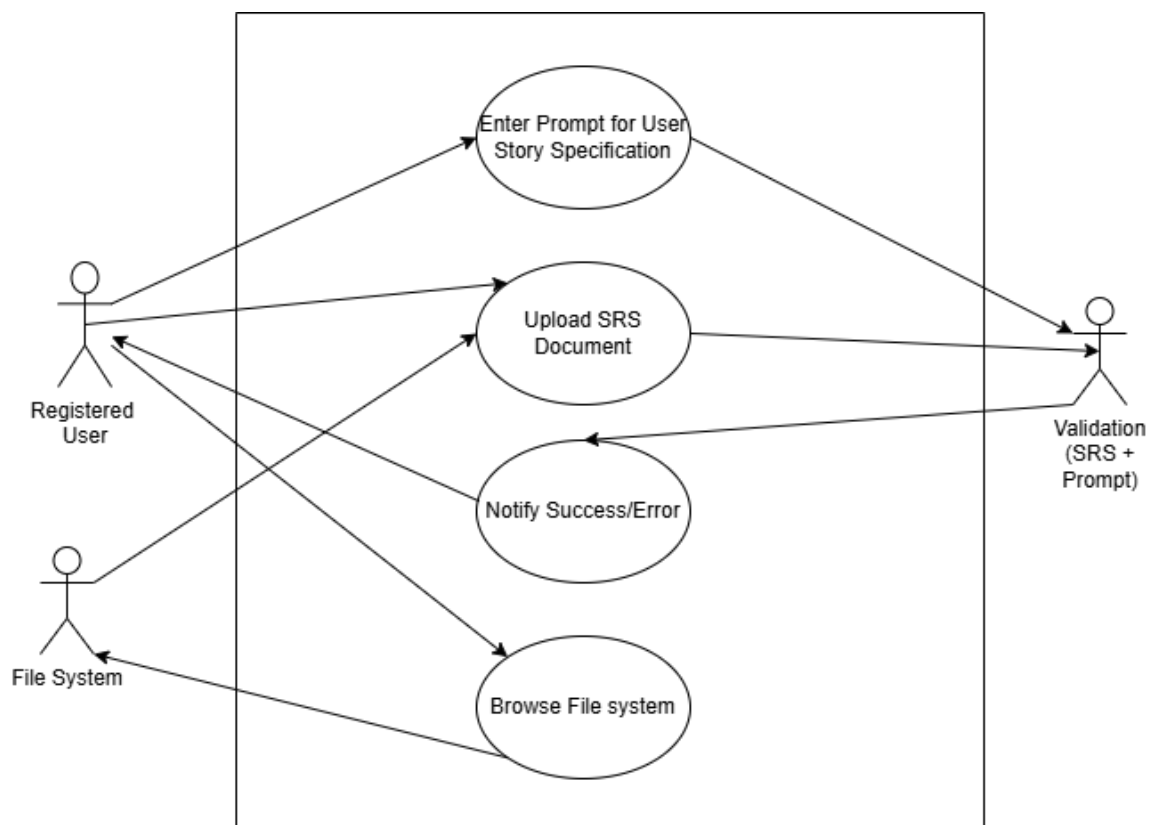


Figure 2.1: Use Case diagram for User Input Module

This module acts as initial interface for gathering all the required information from user which are prompt and SRS document, for generating user stories. The key actions include entering a relevant prompt, browsing local file system to find and upload SRS document and handling response and actions for upload success or failure.

Table 2.1 provides details about high level use cases.

Use Case	Actors	Type	Description
Enter Prompt for User Story Specification	User	Primary	The user provides text input to further clarify requirements of SRS documents.
Upload SRS Document	User, File System	Primary	The user locates the document on the local file system and uploads it.
Notify Success/Error	System	Secondary	After the document is uploaded, the system will perform a check and inform the user if the file upload was successful or if there is a problem.
Browse File System	User, File System	Primary	User navigates and finds the document to upload.

Table 2.1: Use Case Descriptions

Below are detailed use case for this module:

Name	Enter Prompt for User Story Specification
Actors	User
Pre Conditions	• User must be logged in. • The system is ready to receive the prompt.
Post Conditions	Prompt is successfully stored for usage.
Main Success Scenario	• The user navigates to the input form for entering the prompt. • The user types in the desired text prompt. • The user submits the prompt. • The system validates the prompt (e.g., checks for empty input). • The system accepts the prompt and stores it for further processing.
Extensions	Invalid Prompt: If the prompt is invalid (e.g., too short or empty), the system displays an error message and prompts the user to correct it.
Triggers	The user selects the prompt input field and begins typing.
Special Requirements	• The system should validate the input to ensure it's a non-empty string. • The system should support prompts of variable lengths.

Table 2.2: Detail Use Case for Module 1, Use Case 1

Name	Upload SRS Document
Actors	User, File System
Preconditions	• The user must be authenticated and logged into the system. • The user must have the SRS document stored locally on their device.
Postconditions	• The system receives the uploaded SRS document. • The system stores the document for processing or notifies the user of any issues.
Main Success Scenario	• The user selects the option to upload an SRS document. • The system opens the file system for browsing. • The user navigates the file system and selects the appropriate SRS document. • The system accepts the document and begins uploading. • The system notifies the user of a successful upload.
Extensions	Error Handling • If the file format is incorrect or invalid, the system notifies the user of an error and prompts them to retry. • If the document fails to upload due to network or system issues, the system informs the user of the error and suggests trying again later.
Triggers	The user initiates the file upload process by selecting the upload button.
Special Requirements	• The user must upload specific file formats. • The system should handle large file uploads efficiently.

Table 2.3: Detail Use Case for Module 1, Use Case 2

Name	Notify Success/Error
Actors	System
Preconditions	• The user must have attempted an action (e.g., uploading a document or entering a prompt). • The system must have processed the action.
Postconditions	• The user is informed of the success or failure of the action.
Main Success Scenario	• The system detects that the user has submitted an action (e.g., document upload or prompt submission). • The system checks whether the action was completed successfully. • If successful, the system sends a notification confirming the success. • If there is an error (e.g., file upload failure), the system sends an error notification.
Extensions	• Error Message: The system provides details on the nature of the error and suggests corrective actions (e.g., re-uploading the file). • Success Message: The system confirms that the document or prompt was successfully submitted and received.
Triggers	The system sends a notification after processing an action.
Special Requirements	• Notifications should be clear and informative.

Table 2.4: Detail Use Case for Module 1, Use Case 3

Name	Browse File System
Actors	User, File System
Preconditions	• The user is logged into the system. • The system has requested the user to upload a document.
Postconditions	• The user selects a file for upload.
Main Success Scenario	• The user clicks the button to browse the file system. • The system opens a file browser dialog. • The user navigates through directories to locate the desired SRS document. The user selects the file and confirms the selection. • The file is prepared for upload.
Extensions	• File Not Found. • Wrong File Type.
Triggers	The user clicks the "Browse" button to start file navigation.
Special Requirements	• The system should ensure the selected file meets the necessary format requirements before proceeding to upload.

Table 2.5: Detail Use Case for Module 1, Use Case 4

2.1.2 User Story Generation Module

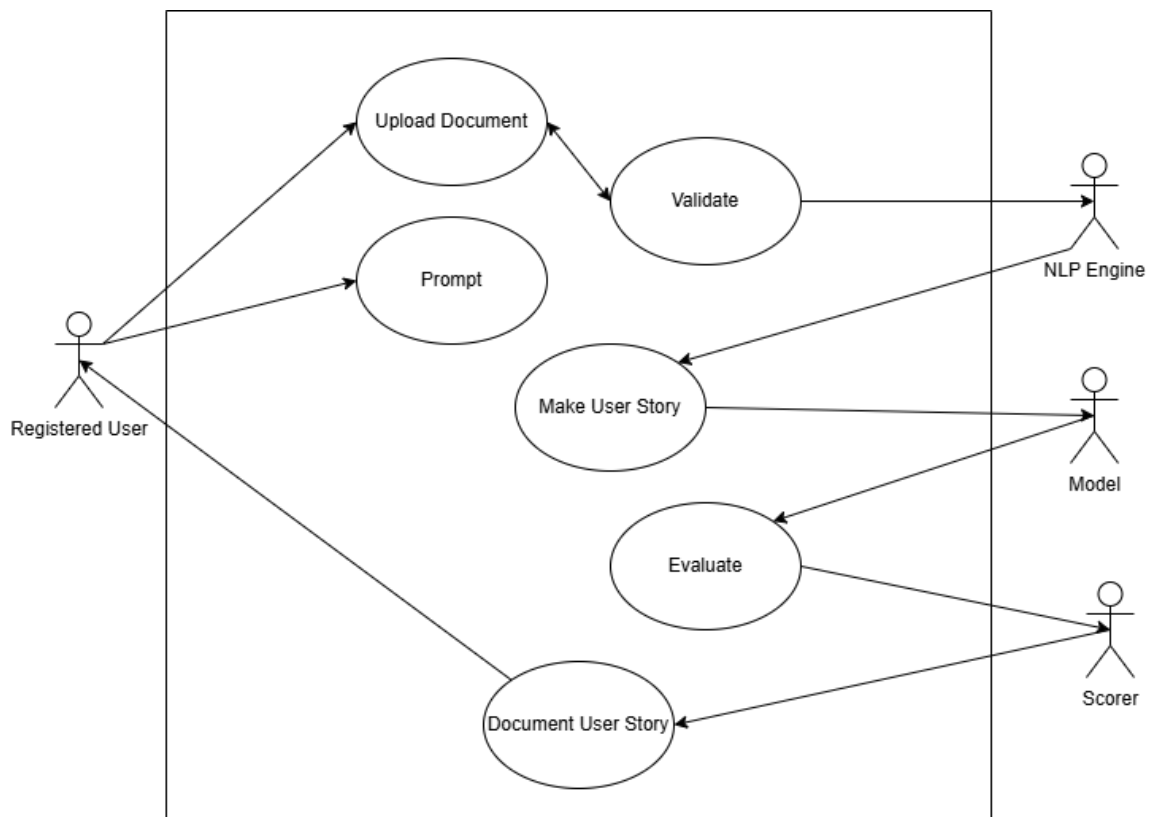


Figure 2.2: Use Case diagram for User Story Generation Module

This module is designed to automate the process of user story generation from prompt and document submitted by user. This module processes inputs, generate user stories, evaluates each story for correctness and stores them for user. It simplifies creation of user stories for product manager by eliminating week of daily meetings and help manager to translate clients requirements to developers.

Table 2.6 provides details about high level use cases.

Use Case	Actors	Type	Description
Upload Document	User	Primary	User uploads document containing information regarding clients product.
Validate	System	Primary	System checks for any errors in document (eg. file format, SRS document format)
Prompt	User, System	Primary	System request prompt from user for further clarification about this document.
Make user story	System	Primary	System generates user stories based on provided data.
Evaluate	System, User	Primary	The generated user stories are reviewed for its correctness.
Document User Story	System	Primary	System finalizes and documents generated user stories and stores it for future use.

Table 2.6: Use Case Descriptions for Module 2

Below are detailed use case for this module:

Name	Upload Document
Actors	• Primary: User • Secondary: System
Pre Conditions	User must have access to system and document must be in valid format.
Post Conditions	The document is uploaded and ready for validation.
Main Success Scenario	• User selects upload option • System presents file explorer to select desired document. • User clicks upload. • Sytem receives documents and stores for processing.
Extensions	• Incorrect file format. • Action failed fue to network issues.
Triggers	User clicks upload button.
Special Requirements	Document must be in .pdf format.

Table 2.7: Detail Use Case for Module 2, Use Case 1

Name	Validate
Actors	System
Pre Conditions	Document must be uploaded successfully.
Post Conditions	Document is validated and ready for generating stories.
Main Success Scenario	• System identifies correct format of document. • System check that all required information is present in document • System confirms that document is error free.
Extensions	If the document is missing essential data, the system prompts the user to re-upload.
Triggers	Validation is triggered automatically after the document is uploaded.
Special Requirements	The system must be able to recognize correct file format and detect incomplete or erroneous data.

Table 2.8: Detail Use Case for Module 2, Use Case 2

Name	Prompt
Actors	• Primary: User • Secondary: System
Pre Conditions	Document must be uploaded and validated.
Post Conditions	System gather all required information.
Main Success Scenario	• The system prompts the user for any missing or unclear details • The user provides the requested input through a prompt.
Extensions	If the user input is unclear or incomplete, the system requests further clarification.
Triggers	The system triggers the prompt when it detects missing or unclear information in the uploaded document.
Special Requirements	The system must be able to ask relevant and context-aware questions based on the document's content.

Table 2.9: Detail Use Case for Module 2, Use Case 3

Name	Make User Story
Actors	Primary: User
Pre Conditions	All necessary data has been uploaded and validated (both document and prompt).
Post Conditions	A set of user stories is generated and ready for evaluation.
Main Success Scenario	• The system parses the document to extract relevant requirements. • Based on the requirements, the system generates user stories. • The user stories are stored in a temporary format for evaluation.
Extensions	If the system encounters ambiguous requirements, it must take action to counter it.
Triggers	The system starts generating user stories once the input document is validated or the prompt is completed.
Special Requirements	• System must generate multiple user stories. • System must ensure that it follows standard format(As a [user], I want to [action], so that [outcome]). • System must generate acceptance criteria for each user story.

Table 2.10: Detail Use Case for Module 2, Use Case 4

Name	Evaluate
Actors	• Primary: System • Secondary: User
Pre Conditions	User Stories must be generated.
Post Conditions	The user stories are accurate and complete.
Main Success Scenario	• The system evaluates the generated user stories against the input requirements. • The system identifies potential inconsistencies or missing details. • System regenerates user stories.
Extensions	If the stories are inaccurate, the system adjusts them based on further analysis or user input.
Triggers	The evaluation process is started immediately after user story generation.
Special Requirements	The system must allow for both automated evaluation and manual review by the user.

Table 2.11: Detail Use Case for Module 2, Use Case 5

Name	Document User Stories
Actors	System
Pre Conditions	User stories have been generated and evaluated.
Post Conditions	The user stories are documented and ready for use by developers.
Main Success Scenario	• The system formats the confirmed user stories in a structured document. • The document is stored in the system for further use or export. • The user is notified that the stories have been documented successfully.
Extensions	If the system encounters an error during the documentation process, it logs the error and notifies the user.
Triggers	The documentation process begins once the user stories are evaluated successfully.
Special Requirements	• The system should allow users to export stories in .pdf formats. • The system must ensure that the documented user stories are easy to access for future reference.

Table 2.12: Detail Use Case for Module 2, Use Case 6

2.1.3 Version Control and Tracking Module

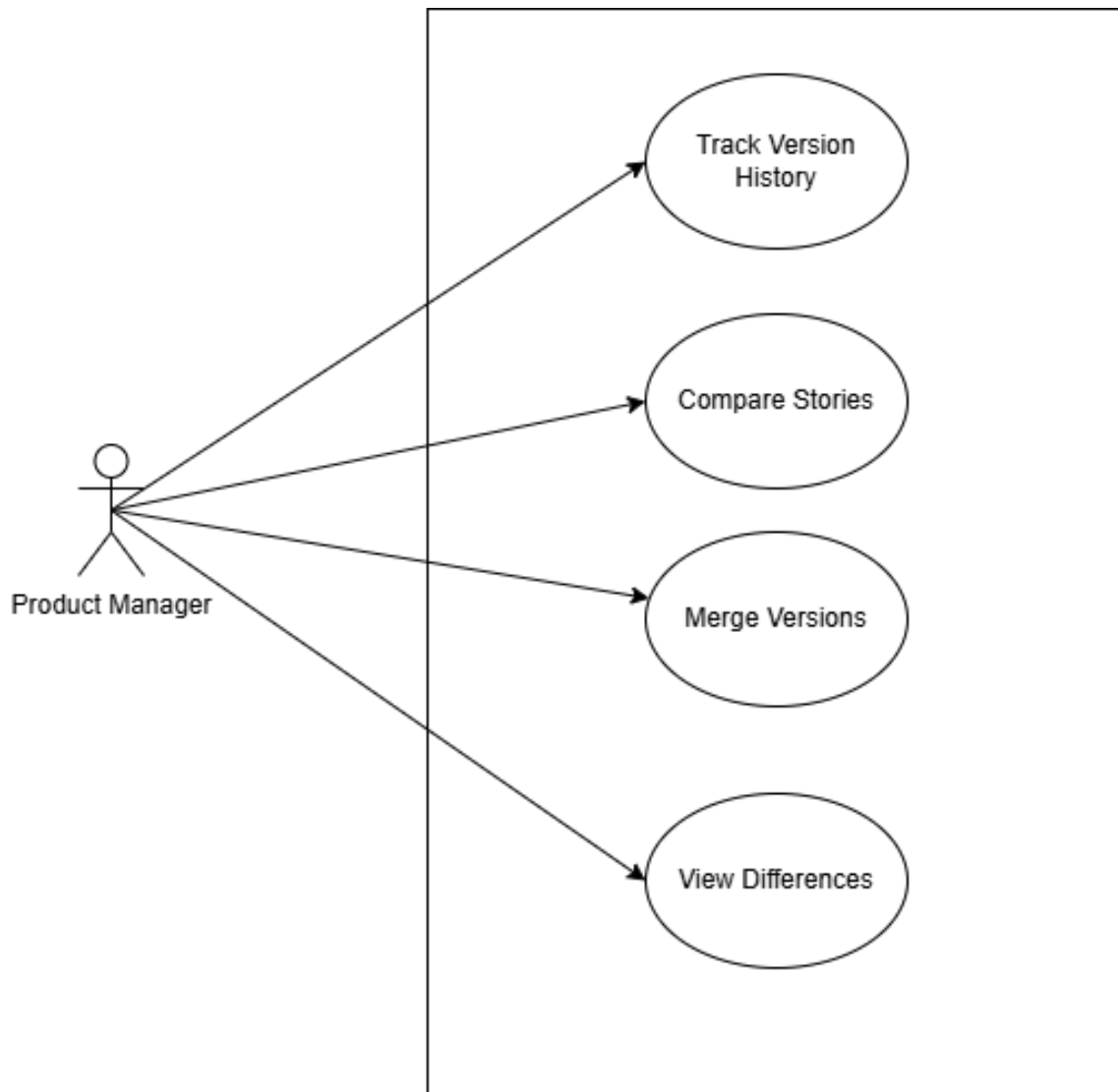


Figure 2.3: Use Case diagram for Version Control and Tracking Module

This module helps developers and product managers track the history of user stories, compare different versions and merge multiple versions into single one. This ensure that changes are reflected for everyone who can access this user stories and each change in stories is logged, tracked, and properly managed through out development process to maintain version consistency.

Table [2.13](#) provides details about high level use cases:

Use Case	Actors	Type	Description
Track Version History	User, System	Primary	Allow user to view different versions of user stories.
Compare Stories	User, System	Primary	Allow user to compare 2 different versions of user stories.
Merge Version	User, System	Primary	Merges 2 different version into single one.
View Differences	User, System	Primary	Highlights differences between different versions of user stories.

Table 2.13: Use Case Descriptions for Module 5

Below are detailed use case for this module:

Name	Track Version History
Actors	User, System
Pre Conditions	The user must have access to the system, and the user story must have versions stored in the system.
Post Conditions	The version history of the selected user story is displayed to the user.
Main Success Scenario	• The user selects a user story. • The system displays a list of versions with timestamps. • The user can browse through and view version details.
Extensions	If no versions are available, the system notifies the user that no version history exists.
Triggers	The user selects the Track History option from the system.
Special Requirements	The system must efficiently store and retrieve version history for quick access.

Table 2.14: Detail Use Case for Module 5, Use Case 1

Name	Compare Stories
Actors	User, System
Pre Conditions	Multiple versions of user stories must be available for comparison.
Post Conditions	The differences between the selected versions of the user stories are displayed.
Main Success Scenario	• The user selects multiple versions to compare. • The system highlights differences between versions. • The user views the comparison results.
Extensions	If the stories are identical, the system notifies the user that no differences exist.
Triggers	The user selects the "Compare Stories" option and selects versions to compare.
Special Requirements	The system must accurately compare and highlight differences between versions.

Table 2.15: Detail Use Case for Module 5, Use Case 2

Name	Merge Versions
Actors	User, System
Pre Conditions	Multiple versions of a user story must be available for merging.
Post Conditions	A new, merged version of the user story is created.
Main Success Scenario	• The user selects multiple versions to merge. • The system prompts the user to resolve conflicts. • The system merges the versions and creates a new version.
Extensions	If conflicts arise, the system prompts the user for conflict resolution.
Triggers	The user selects the "Merge Versions" option after selecting the versions.
Special Requirements	The system must efficiently handle conflicts during the merge process and guide the user.

Table 2.16: Detail Use Case for Module 5, Use Case 3

Name	View Differences
Actors	User, System
Pre Conditions	The user must have access to different versions of a user story for comparison.
Post Conditions	The differences between the selected user stories are displayed to the user.
Main Success Scenario	• The user selects two versions to compare. • The system highlights the differences between the selected versions. • The user views the highlighted changes.
Extensions	If no differences are found, the system notifies the user.
Triggers	The user selects the "View Differences" option for a user story.
Special Requirements	The system must clearly highlight and present differences in an understandable format.

Table 2.17: Detail Use Case for Module 5, Use Case 4

2.1.4 Security and Authentication Module

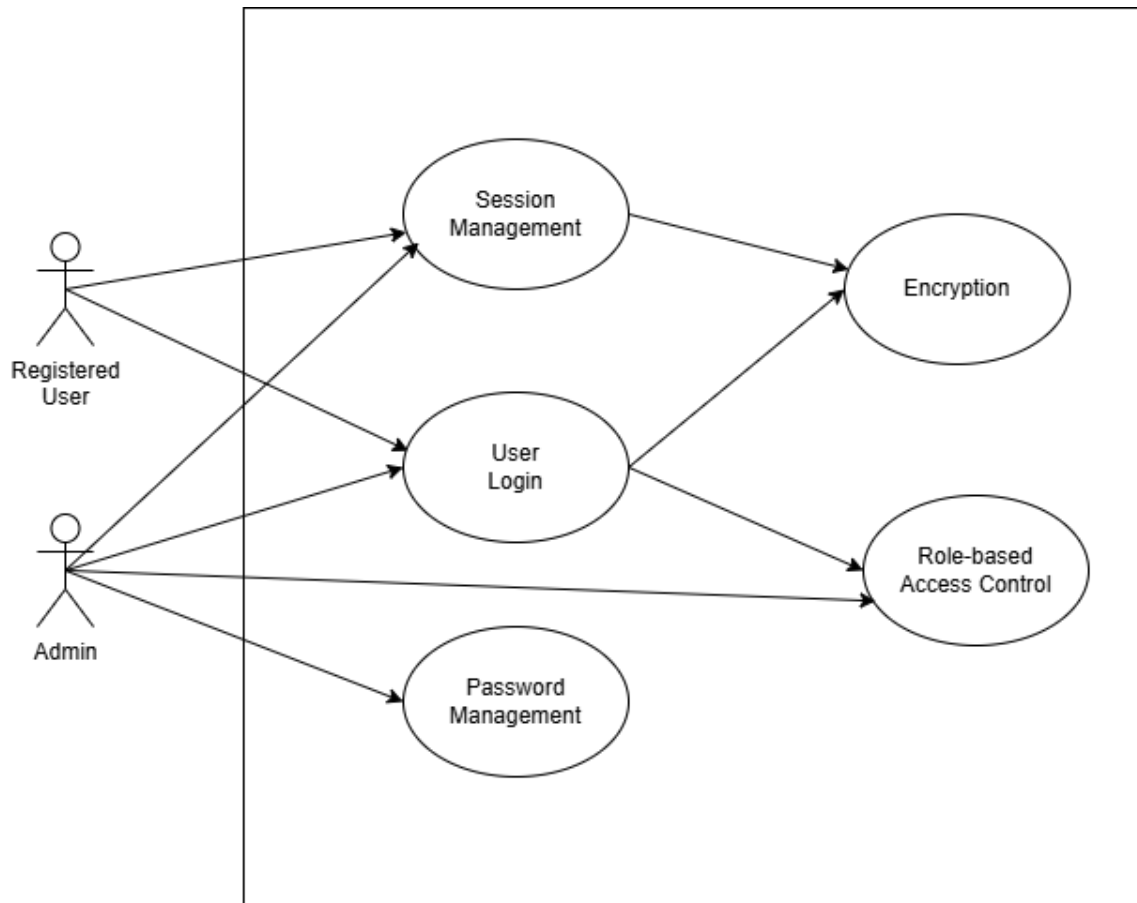


Figure 2.4: Use Case diagram for Security and Authentication Module

This module represents how our product will authenticate users to maintain security of our product. The main functionalities include managing sessions, handling user logins, password management and Role-based Access Control. These are essential for ensuring secure access to systems, maintaining confidentiality and integrity, and allow roll-specific permissions.

Table [2.18](#) provides details about high level use cases:

Use Case	Actors	Type	Description
Session Management	User	Primary	Manages the user's session state after login, including timeout and renewal mechanisms.
User Login	User	Primary	Authenticates user credentials to provide access to the system.
Password Management	User, Admin	Primary	Allows the user to change or reset their password, with admin assistance if needed.
Encryption	System	Primary	Secures data transmission and storage through encryption techniques.
Role-based Access Control (RBAC)	Admin, System	Primary	Assigns roles to users and restricts access based on defined permissions.

Table 2.18: Use Case Descriptions for Module 6

Below are detail use case for this module:

Name	Session Management
Actors	User
Pre Conditions	The user is logged in successfully.
Post Conditions	The user's session is maintained, renewed, or expired based on interaction or timeout.
Main Success Scenario	• The user logs in, and a session is initiated. • The system manages the session, renewing it on activity or expiring it after inactivity. • The user continues using the system as long as the session is active.
Extensions	If the session expires, the user is prompted to log in again.
Triggers	User logs in, or session timeout occurs.
Special Requirements	Session should be renewed seamlessly during user activity.

Table 2.19: Detail Use Case for Module 6, Use Case 1

Name	User Login
Actors	User
Pre Conditions	User is not logged into the system.
Post Conditions	User is successfully authenticated and logged into the system.
Main Success Scenario	• User enters credentials. • The system verifies the credentials. • If successful, the user gains access to the system.
Extensions	If credentials are invalid, the system prompts the user to re-enter them.
Triggers	The user submits login credentials.
Special Requirements	Authentication should be secure, possibly using multi-factor authentication (MFA).

Table 2.20: Detail Use Case for Module 6, Use Case 2

Name	Password Management
Actors	User, Admin
Pre Conditions	User is registered in the system.
Post Conditions	The user's password is updated or reset successfully.
Main Success Scenario	• User initiates a password change or reset. • The system verifies the user (through security questions or email). • The user sets a new password successfully.
Extensions	If verification fails, the system notifies the user and retries.
Triggers	The user requests a password change or reset.
Special Requirements	Passwords should meet security standards (e.g., length, complexity).

Table 2.21: Detail Use Case for Module 6, Use Case 3

Name	Encryption
Actors	System
Pre Conditions	Data is ready to be transmitted or stored.
Post Conditions	Data is encrypted and protected from unauthorized access.
Main Success Scenario	• The system encrypts data using a secure algorithm. • Encrypted data is transmitted or stored securely.
Extensions	If encryption fails, the system retries or notifies the administrator.
Triggers	Data is about to be transmitted or stored.
Special Requirements	Encryption algorithms should follow industry standards

Table 2.22: Detail Use Case for Module 6, Use Case 4

Name	Role-based Access Control (RBAC)
Actors	Admin, System
Pre Conditions	Users and roles are defined in the system.
Post Conditions	Users have access to resources that are allowed by admin based on their role.
Main Success Scenario	• Admin assigns roles to users. • The system enforces access control based on roles. • Users access resources based on their assigned role permissions.
Extensions	If role assignment fails, the system notifies the admin and retries.
Triggers	Admin assigns or modifies a user's role.
Special Requirements	Access control policies should be scalable for multiple roles and permissions.

Table 2.23: Detail Use Case for Module 6, Use Case 5

2.2 Functional Requirements

2.2.1 User Input

1. The module shall provide functionality to allow users to upload SRS (Software Requirements Specification) documents.

Details:

- The module should support document formats such as .doc, .docx, and .pdf.
 - Users should be able to browse their local file system and select the SRS document to upload.
 - The system must notify users upon successful upload or in case of failure with appropriate error messages.
2. The module shall provide a prompt input field where users can specify the type of user stories they want to generate.

Details:

- The prompt field should support text input, allowing users to specify requirements for functional, non-functional, or specific sections of the SRS.
- The interface should guide the users with examples or suggestions on how to write effective prompts.

- The system must process and store the prompt for subsequent user story generation.
3. The module shall validate the uploaded SRS document and the prompt input to ensure they meet the required format and structure.

Details:

- The system must check the file type of the uploaded document and reject unsupported formats with an appropriate error message.
- The prompt field must have validation rules to ensure the prompt meets the minimum and maximum length requirements.
- The system should provide real-time feedback to users if the uploaded document or entered prompt does not meet the required criteria.
- The validation process must be efficient to ensure a smooth user experience.

2.2.2 Natural Language Processing (NLP) Engine

1. The NLP engine shall extract key requirements from the uploaded SRS document.

Details:

- The engine should identify and extract functional and non-functional requirements from the SRS document.
 - It should recognize and tag different components such as goals, constraints, and assumptions.
 - The engine must handle diverse document structures and terminologies to accurately capture the requirements.
2. The NLP engine shall understand the context of the SRS document to accurately interpret its content.

Details:

- The engine must be capable of handling complex language constructs and technical jargon to derive meaningful insights.
3. The NLP engine shall interpret the user's prompt to generate tailored user stories.

Details:

- The engine should process the user's prompt to understand specific requirements for user stories (e.g., focus on particular sections, functional vs. non-functional).
- It must translate technical language and detailed requirements into clear, user-friendly language suitable for user stories.
- The generated user stories should be coherent, contextually accurate, and aligned with the user's prompt specifications.
- The engine should provide feedback to users if the prompt is ambiguous or insufficient for generating meaningful user stories.

2.2.3 User Story Generation

1. The module shall automatically generate user stories from the data extracted by the NLP engine.

Details:

- The system should adhere to a predefined format for user stories, such as "As a [role], I want [goal], so that [benefit]".
 - The module must ensure that each user story is coherent, concise, and follows the standard user story structure.
 - The generation process should be efficient, providing quick results to the user.
2. The module shall be capable of generating user stories for both functional and non-functional requirements.

Details:

- The system should distinguish between functional and non-functional requirements and generate appropriate user stories for each.
 - Functional requirements user stories should focus on describing the actions and features of the system.
 - Non-functional requirements user stories should address performance, security, usability, and other quality attributes.
 - The module must ensure that all relevant requirements extracted by the NLP engine are covered in the generated user stories.
3. The module shall allow customization of user stories based on the user's prompt.

Details:

- The module should interpret the user's prompt to tailor the user stories according to the specific instructions provided.
- It should allow customization in terms of focus areas, level of detail, and specific sections or types of requirements.
- The system must ensure that the generated user stories align with the user's prompts while maintaining coherence and clarity.

2.2.4 Story Review and Editing

1. The module shall allow users to review and edit the generated user stories.

Details:

- Users should be able to modify the text of the user stories to better fit their needs or correct any inaccuracies.
 - The interface should provide a user-friendly text editor with basic formatting options (e.g., bold, italic, underline).
 - Changes made by users should be saved and reflected in the final user story output.
 - The system must provide version control to track changes and allow users to revert to previous versions if needed.
2. The module shall provide mechanisms for users to give feedback on the generated user stories and suggest adjustments to the generation rules.

Details:

- Users should be able to submit feedback on specific user stories, including suggestions for improvements or noting any issues.
- The system should collect and analyze feedback to identify common patterns or areas for improvement in the story generation process.
- The interface should allow users to propose new rules or modifications to existing rules for generating user stories.
- The module should facilitate communication between users and developers to ensure feedback is addressed and incorporated into future updates.

2.2.5 Version Control and Tracking

1. The module shall provide functionality to track all changes made to user stories, capturing each version as a separate entity.

Details:

- The system should create a new version of a user story every time it is edited and saved.
 - Metadata for each version, including the author of the changes, timestamp, and a brief description of the changes, must be stored.
 - A version history view, listing all versions of a user story in chronological order, should be available.
 - Users should have the ability to revert to any previous version of a user story.
2. The module shall provide functionality for users to compare different versions of a user story to identify changes.

Details:

- A comparison tool should be available to highlight differences between two selected versions of a user story.
- The tool should display changes in a side-by-side or inline format for easy comparison.
- Users must be able to select any two versions from the version history for comparison.
- Additions, deletions, and modifications between versions should be highlighted.

2.2.6 Security and Authentication

1. The module shall provide functionality for user authentication and login, ensuring that only authorized personnel can access the system.

Details:

- The system must require users to provide valid credentials (username and password) for login.
- The module should support multi-factor authentication (MFA) to enhance security.

- Users must be notified of successful login or authentication failure with appropriate error messages.
 - The system should log all login attempts, including successful and failed attempts, for auditing purposes.
2. The module shall implement role-based access control (RBAC) to manage user permissions and ensure that users only have access to the resources they are authorized to access.

Details:

- The system must allow administrators to define and manage user roles.
- Each role should have specific permissions associated with it, controlling access to different parts of the system.
- Users should be assigned roles based on their responsibilities and level of access required.
- The system must enforce role-based permissions at all times, preventing unauthorized access.

2.3 Non-Functional Requirements

The non-functional requirements for this system define the critical quality attributes that will ensure the system's reliability, performance, and overall user experience. These requirements are measurable, ensuring the system meets specific performance benchmarks, scalability expectations, security standards, and user accessibility needs.

2.3.1 Performance

The system must deliver rapid processing to facilitate real-time usage. From processing the input document and prompt to generating the users stories and then scoring generated user stories, the entire process must be fast and precise. An estimated time of 10-15 seconds is desirable. The system allow simultaneous users activities without any observable delay or significant performance deterioration. This performance metric ensures that the system remains efficient, even during periods of high demand.

2.3.2 Scalability

As the system is expected to handle growing user bases over time, it must be scalable. The system should support multiple concurrent users, adapt to increased workload without

performance bottlenecks. This scalability should be achieved through horizontal scaling mechanisms, such as adding more servers or cloud-based resources when necessary. The system architecture must be structured to accommodate an increased volume of traffic and/or document size, maintaining optimal performance.

2.3.3 Security

Data security is paramount for this system, given that it handles sensitive project documentation and proprietary information. All stored data, including SRS documents and generated user stories, must be encrypted using AES-256 encryption, ensuring maximum data protection. In addition, the system must implement secure user authentication methods, following OAuth 2.0 standards to protect user accounts and access controls. Role-based access control (RBAC) will further ensure that different types of users (e.g., admins, developers, and reviewers) have appropriate levels of access, preventing unauthorized data manipulation.

2.3.4 Usability

For the system to be widely used/accepted, it must prioritize user friendliness. The user interface will be designed to be intuitive and user-friendly, ensuring users can easily upload SRS documents and prompts, even without technical knowledge. Compliance with WCAG 2.1 accessibility standards will be mandatory. Error handling within the system must be comprehensive, providing users with clear, actionable feedback to correct input mistakes without external support, thereby ensuring a seamless user experience.

2.3.5 Reliability

The system is expected to function continuously with minimal interruptions. To this end, it will maintain a service uptime of 90%, ensuring availability for users around the clock. In the event of a system failure, automatic failover mechanisms must be in place to restore services within 1-10 minutes of downtime. Additionally, daily backups of user data to safeguard against data loss or corruption will be implemented, ensuring the ability to restore the system to a working state without significant data loss.

2.3.6 Maintainability

The system's architecture must be designed for easy maintenance and future updates. The codebase should adhere to SOLID design principles, promoting modularity and ease of

change. Allowing developers to make adjustments or implement upgrades without significant rewrites. Scheduled updates and patches must be deploy-able with minimal disruption, keeping system downtime to less than 1 hour. Moreover, comprehensive system logs will track system performance and errors, ensuring that maintenance tasks, such as troubleshooting and diagnostics, can be completed efficiently.

2.4 Domain Model

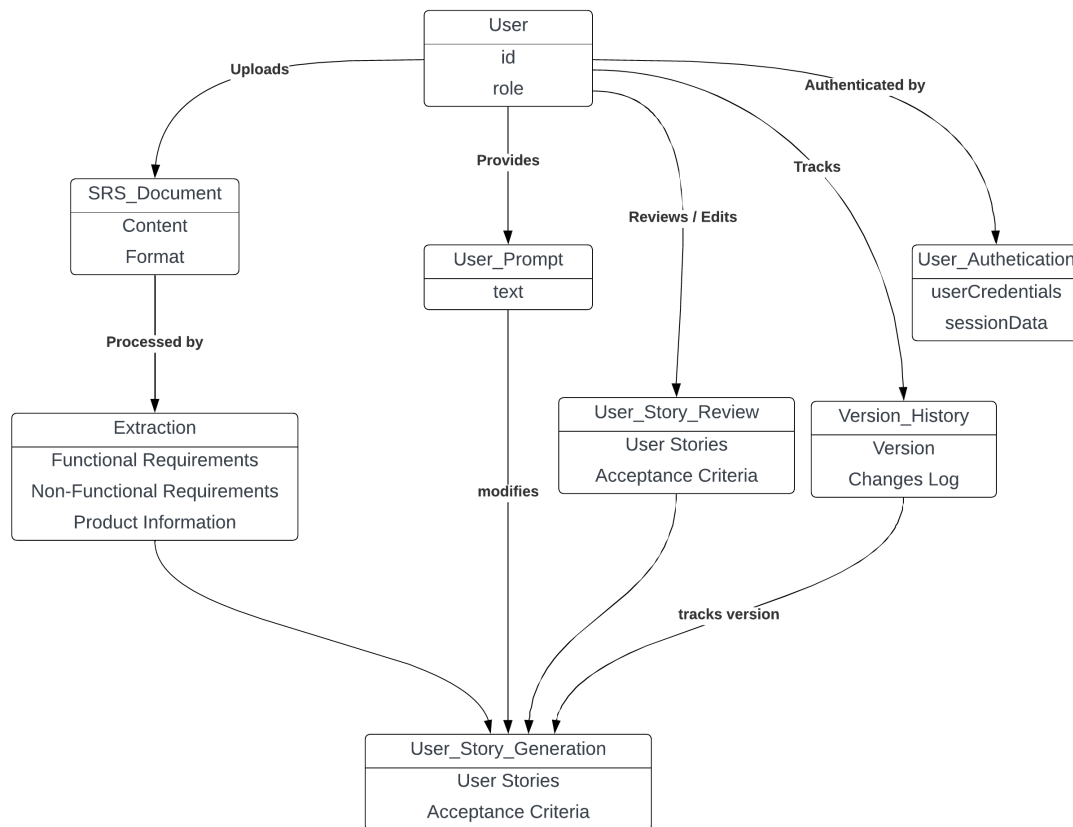


Figure 2.5: Domain Model Diagram

Fig 2.5 is representation of how each main component interacts with each other in order to generate user stories. At core, User, will provide his/her credential to User_Authentication module which will verify if that user exists in our database. Based on his role user will be allowed to upload a Software Specification Document which must be following a specific format. That document goes through extraction module which identifies and store function requirements, non-functional requirements and other important details regarding product.

Along with document, user also provides a prompt that will help our model to better

understand their requirements. User_Story_Generation module takes all the extracted information along with user prompt and generates user stories along with their acceptance criteria. Once generated, user will be able to review and edit all user stories with the help of User_Story_Review module and each change is reflected in changes log which is managed by Version_History module.

Chapter 3

System Overview

3.1 Architectural Design

3.1.1 Box and Line Diagram

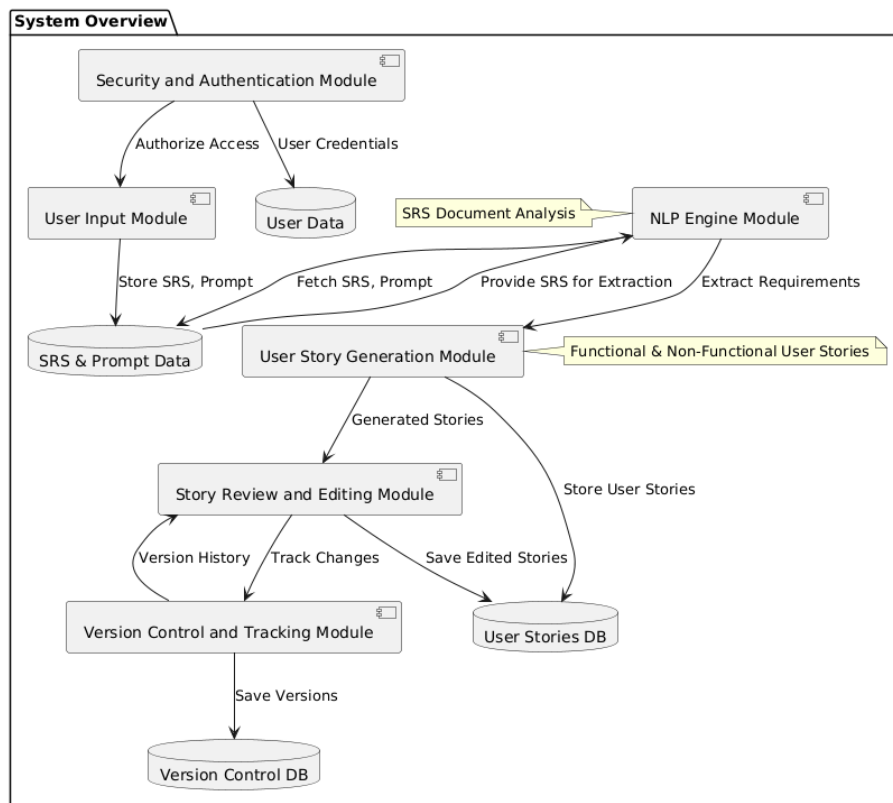


Figure 3.1: Box and Line diagram for complete architecture

Box and line diagram in Figure 3.1 represent how different modules interact with each other to achieve the system's overall functionality. Security and Authentication module is responsible for authentication of each user. Once verified, users can interact with application via user input module where they submit SRS document & prompts. These inputs are stored in database for further processing.

The SRS document analysis step involves extracting necessary details from document such as functional and non-functional requirements with the help of NLP Engine module. This process is crucial, as extracted requirements act as primary input for generation of user stories along with textual prompt given by user.

Once user stories are generated, they go through Story Review and Editing module where user can view and make modifications according to his/her needs. This module is also connected with version history which allows users to track changes in user stories. All edited user stories are stored in Database.

To manage different versions of user stories, the system uses Version Control and Tracking Module. This module makes sure that each iteration of user stories are recorded for maintenance of history of changes in chronological order. The architecture ensures that user stories are not only generated efficiently but also stored securely with robust tracking system.

3.1.2 Model, View and Controller (MVC) Diagram

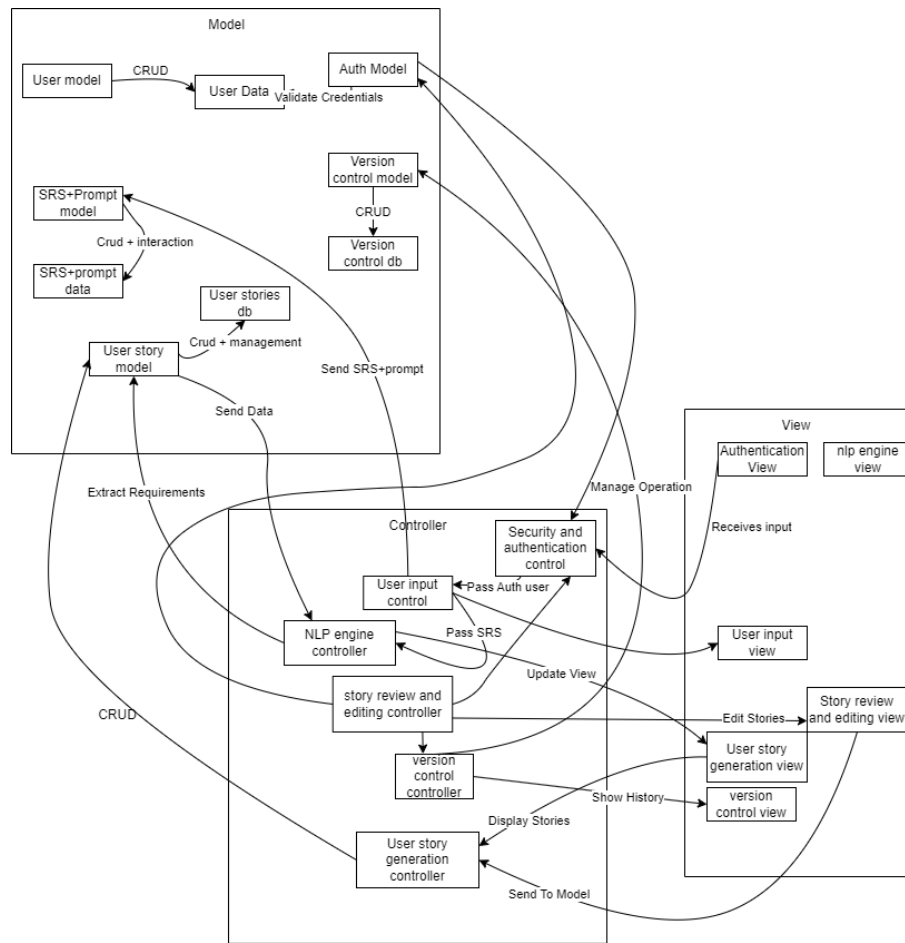


Figure 3.2: MVC diagram for complete architecture

The MVC architecture divides the SprintCraft application into three layers: Model, View, and Controller. The Model manages data, such as users, SRS, and user stories, along with version control. The View handles user interaction interfaces, including story generation, editing, and version history. The Controller acts as a bridge, processing inputs and updating the Model and Views accordingly. Summary for SprintCraft: The MVC architecture for SprintCraft organizes the app into three components. The Model manages data and logic, handling user stories, SRS data, version control, and authentication. The View presents user interfaces for input, story generation, editing, and version history. The Controller processes user actions, updating both the Model and View to ensure smooth user story generation and management. This architecture simplifies maintenance and scalability.

3.2 Design Models

3.2.1 Module 1: User Input Module

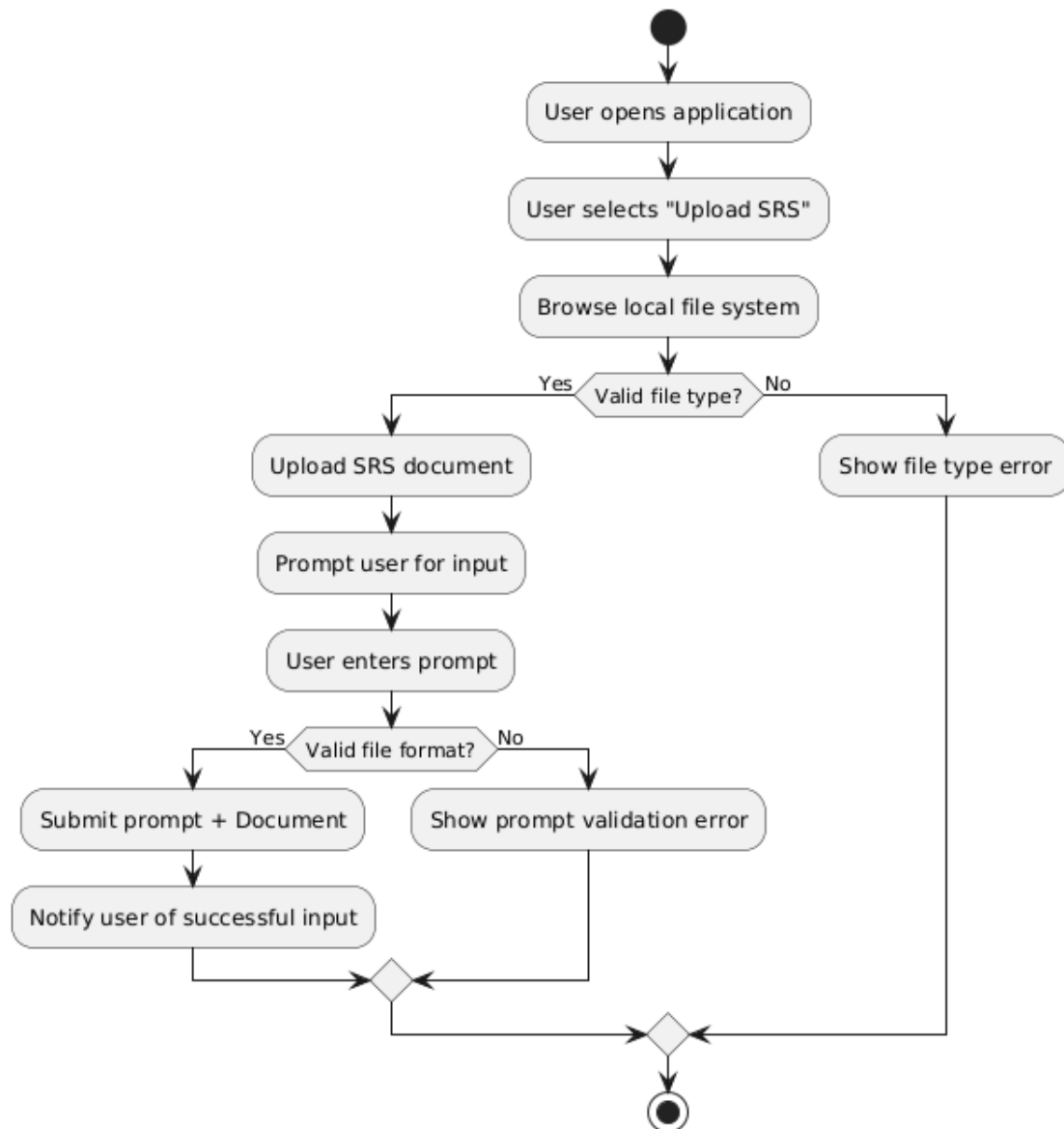


Figure 3.3: Module 1: Activity Diagram

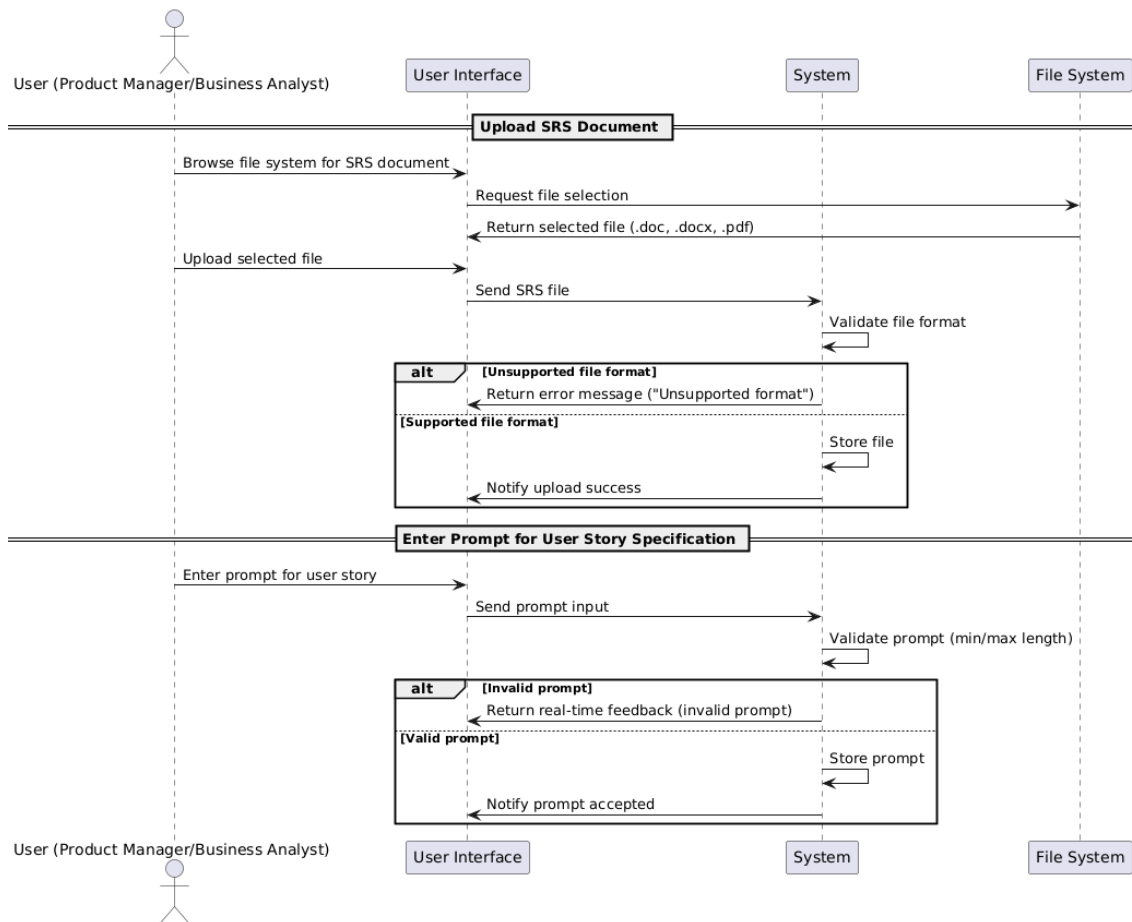


Figure 3.4: Module 1: System Sequence Diagram

The diagrams illustrates how user interacts with application to upload a Software Requirement Specification (SRS) and provide prompt for generating user stories. Both Figures represents how user interacts with system and highlight validation checks and error-handling mechanisms.

Process begins when user open the application and clicks "Upload SRS" button. This will allow users local system to display a window so that user can navigate to desired file he/she wants to submit. The system will check if file extension is valid or not (eg. pdf). If file type is invalid, application will notify user by displaying error message that file type is invalid and ask user to upload again. If it is valid, then document is uploaded and stored on server successfully and system will proceed to ask user to enter a prompt if required. After uploading system will ask user to enter prompt which will act as additional input for generating user stories. Upon submitting, system will initiate validation check. System checks if prompt meet predefined criteria like minimum and maximum length, appropriate formatting etc. System will also validate if content in file is valid or not. Application will try to extract Functional and Non-functional requirements.

Process ends when system notifies user that both prompt and document are valid. The diagrams show the flow of interactions between user and system, showing how errors are handled during whole process. Figure 3.3 provides high level representation of whole process while Figure 3.4 breaks down the internal interactions between user interface and system.

3.2.2 Module 2: Natural Language Processing (NLP) Engine

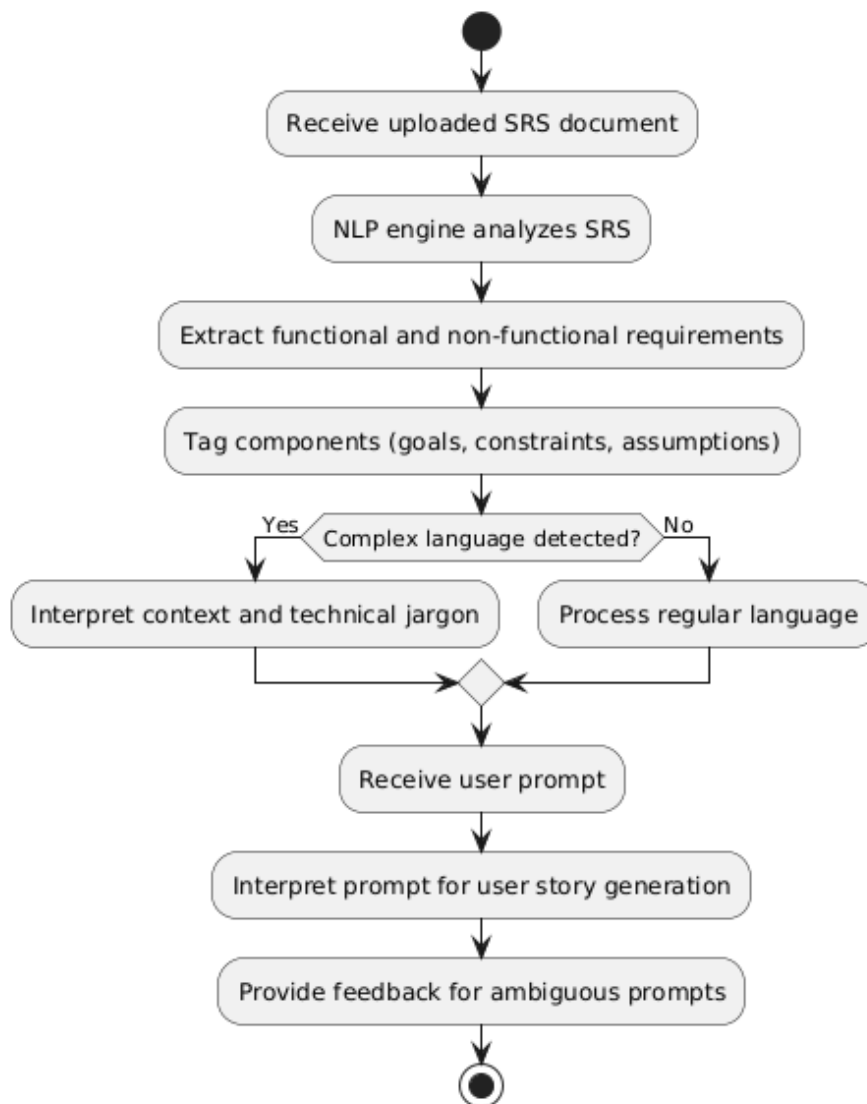


Figure 3.5: Module 2: Activity Diagram

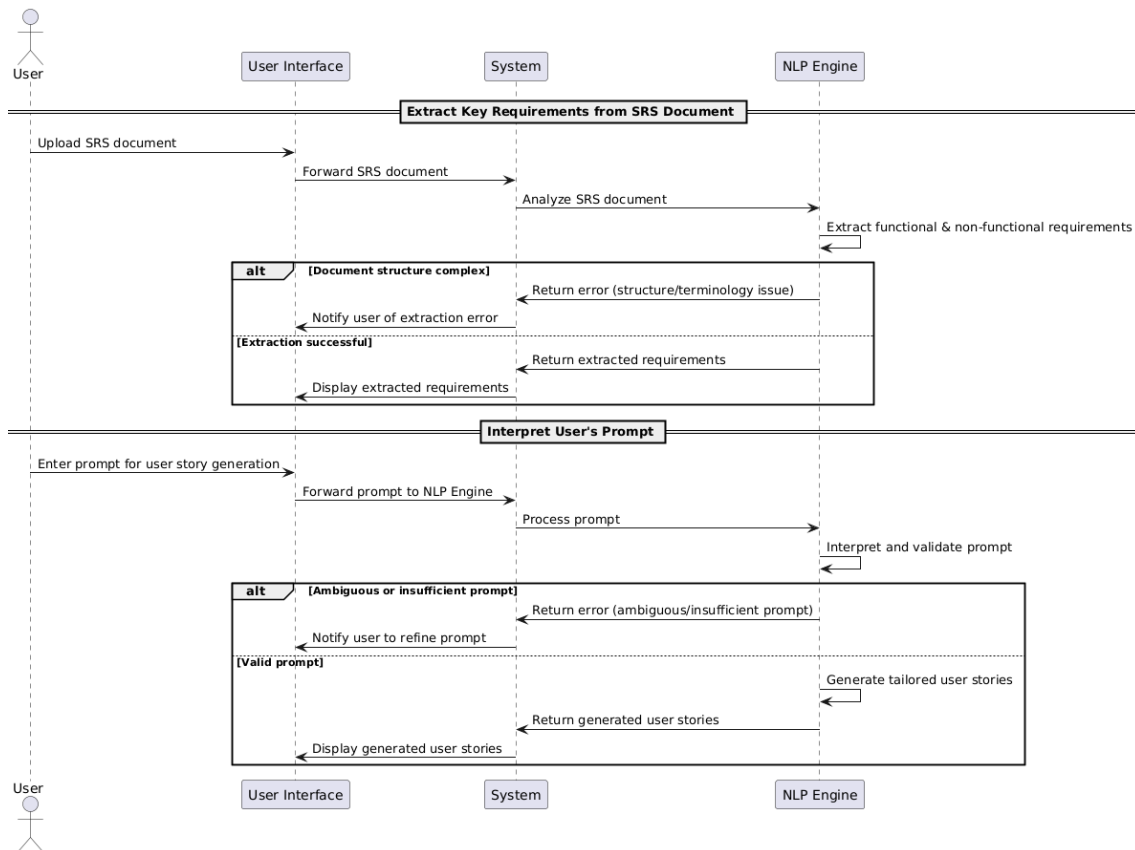


Figure 3.6: Module 2: System Sequence Diagram

NLP Engine, as shown in Figure 3.5 and Figure 3.6 follow a structured workflow that shows how information in document is extracted. Upon receiving document, this engines skims through whole document in order to gather functional and non-functional requirements. During this process, it also tags components as goal, constraints etc which are crucial for understanding. If document contains technical jargon, the system attempts to interpret its meaning and decode its complexity.

Once analysis is complete, it will receive user prompt. If the prompt contains sufficient information and is clear, the system moves ahead with interpreting it for user story generation but if prompt is ambiguous and doesn't align with details extracted from document, user will be notified about this ambiguity.

System Sequence Diagram (Fig. 3.6) provides further details about how this interaction will work by showing responses between modules. If system is unable to understand document's complex structure, it will notify user. Similarly, if there is ambiguity or if information is insufficient in prompt, user is once again notify until both conditions are met. This unified description shows how the NLP engine facilitates requirement extraction and user story generation while providing feedback loops for handling errors or ambiguities at different stages.

3.2.3 Module 3: User Story Generation Module

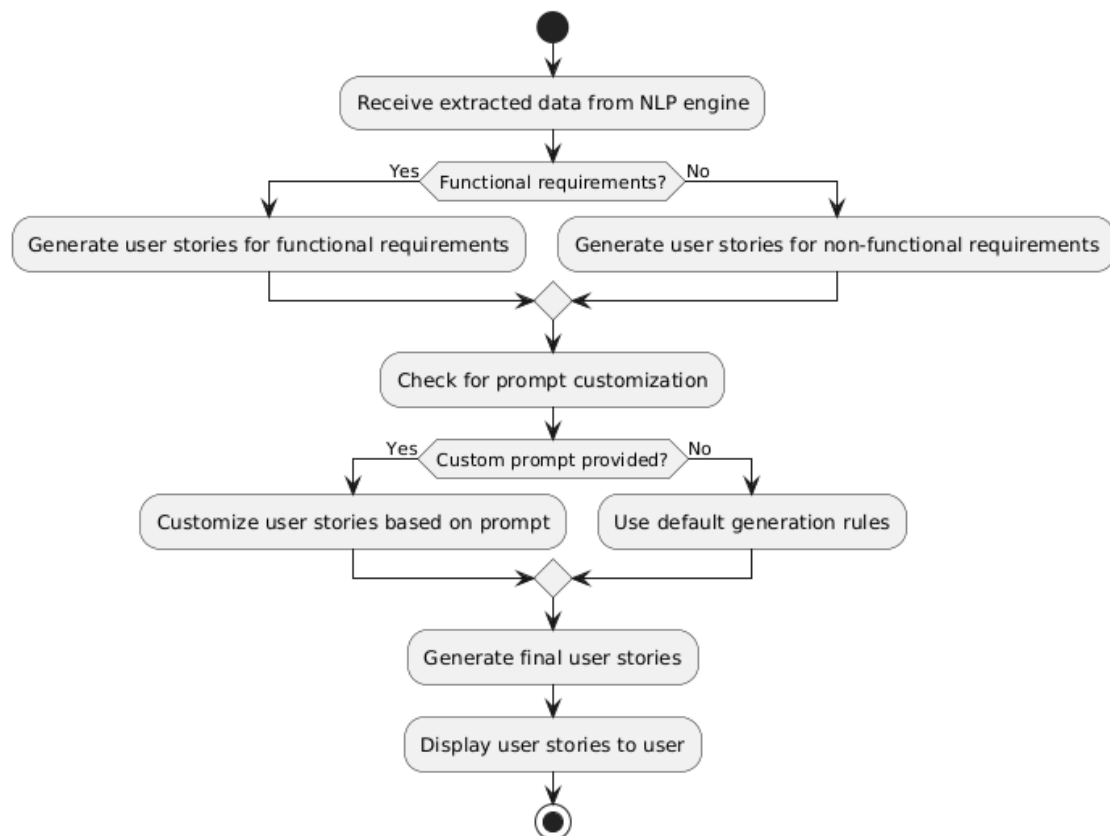


Figure 3.7: Module 3: Activity Diagram

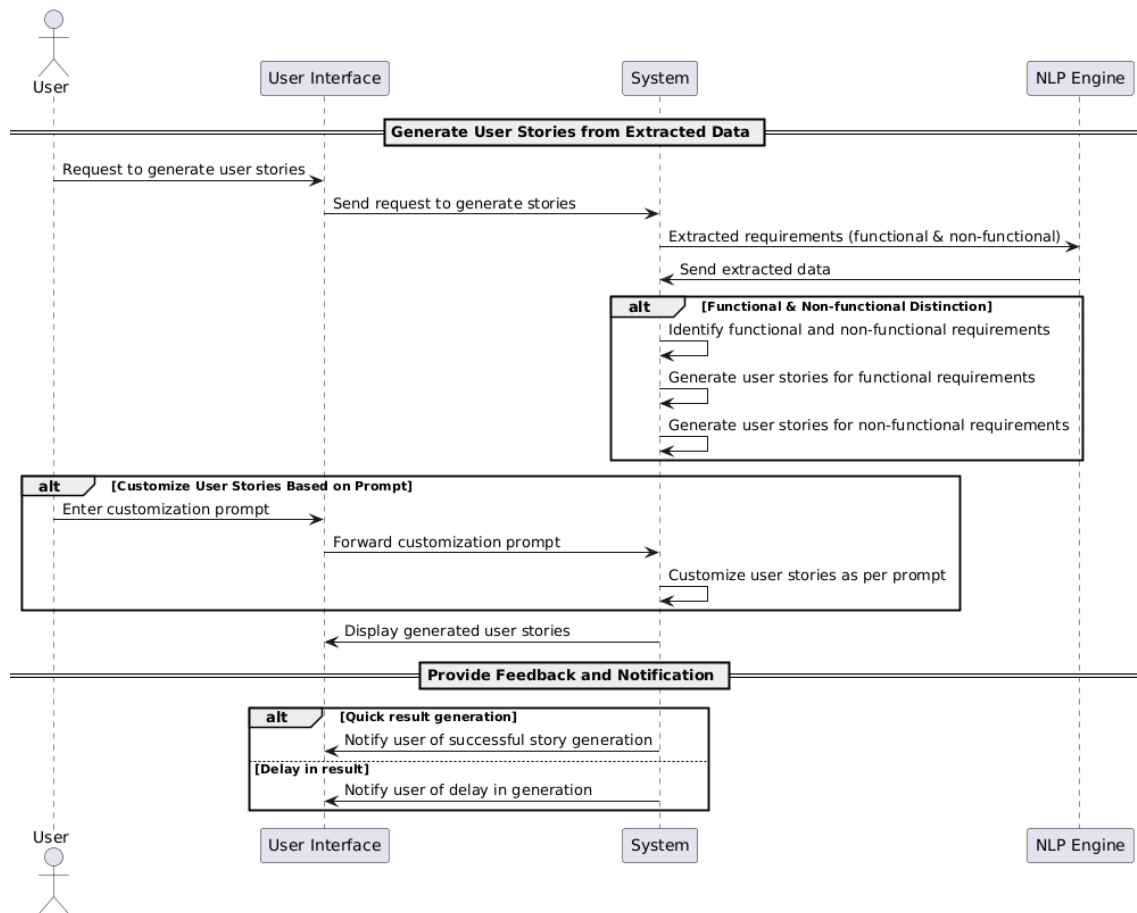


Figure 3.8: Module 3: System Sequence Diagram

Both Figure 3.7 and Figure 3.10 outlines flow of generating user stories from functional and non functional requirement which were extracted in previous module.

When user initiates generation process, system requests all the gathered information from NLP engine. In the Figure 3.10, it is shown that the system distinguishes between two types of requirements, and generates user stories accordingly. This is further clarified in the activity diagram where separate paths are followed for functional and non-functional requirements.

After generation, system check if user has provided a custom prompt or not. If yes, then system adjusts story according to what is required in prompt. If not, then default generation rules are applied. This is represented in both of the provided figures. Lastly, the system notifies the user about the story generation, either confirming success or informing them of delays if any.

3.2.4 Module 4: Story Review and Editing Module

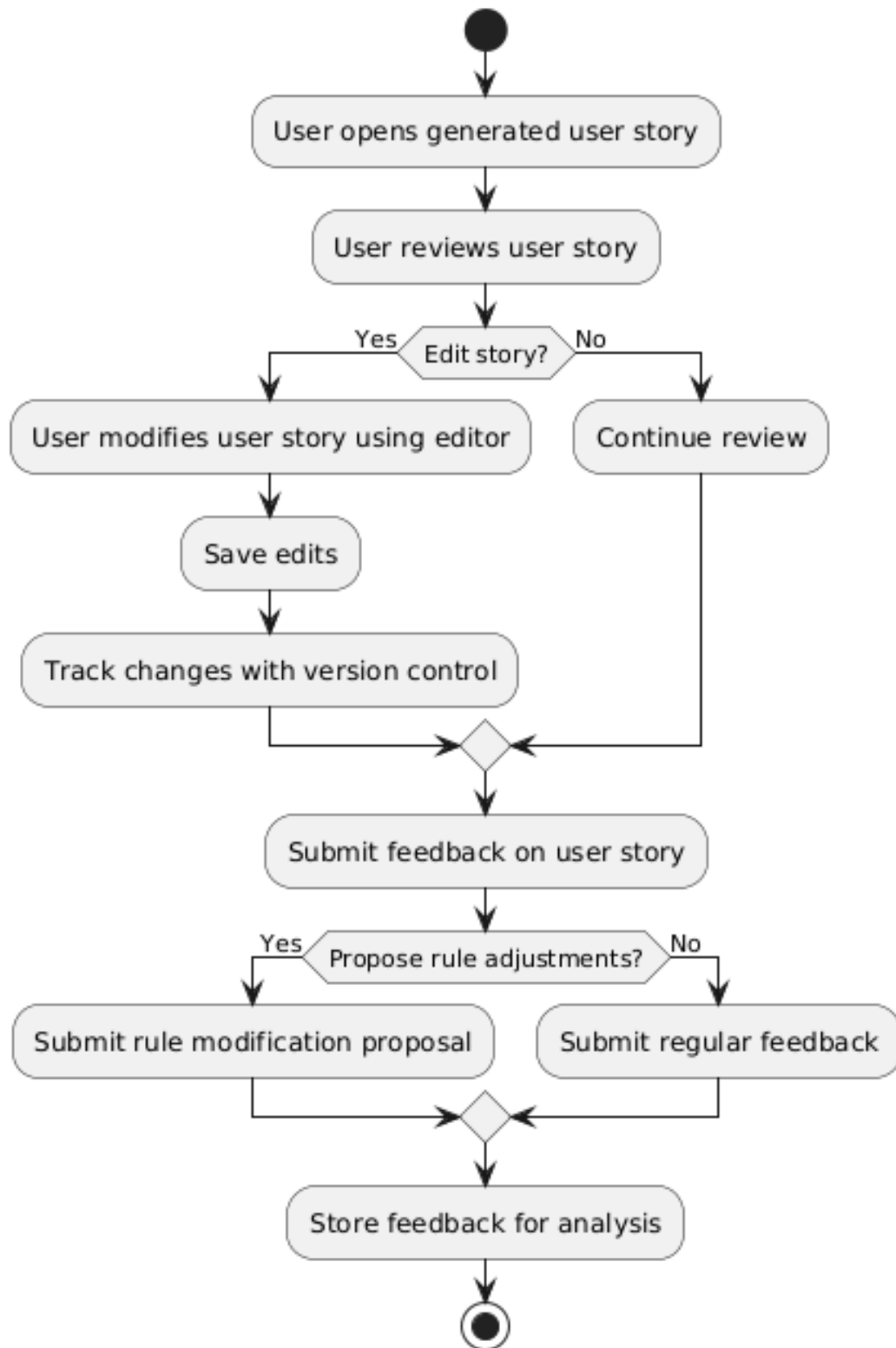


Figure 3.9: Module 4: Activity Diagram

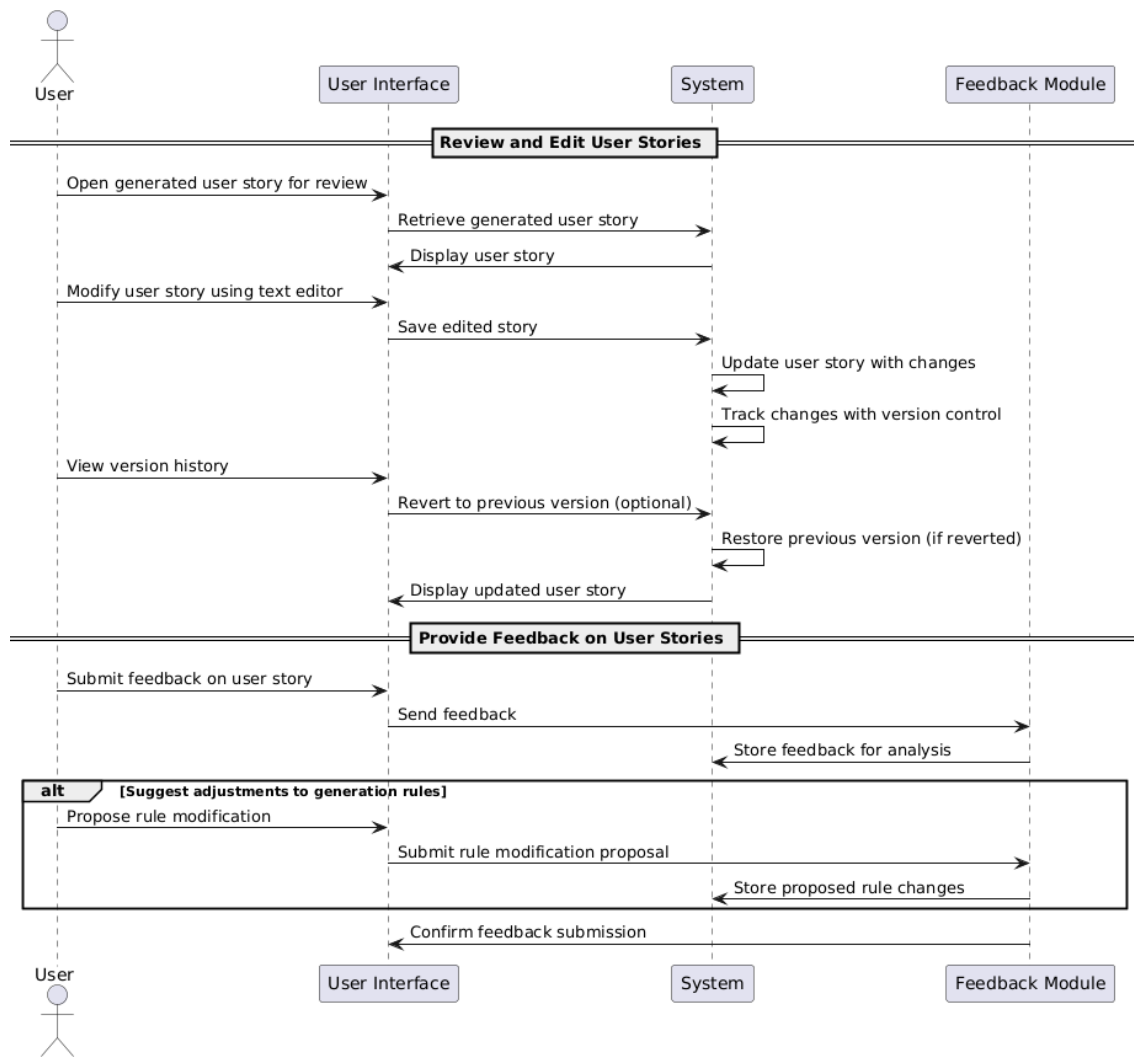


Figure 3.10: Module 4: System Sequence Diagram

This module follows a structured workflow that allow users to review and edit user stories. This process starts when user open a generated user story for evaluation. The user reviews the content and determines whether changes are required or not. If user decides to make changes, user is redirected to editor where they can modify. Once changes are made, the user will save edits which will be reflected to version control system where details of each edit will be recorded.

After review, user is asked to submit feedback on the user story. Here they have choice to either submit rule modification proposal or just submit regular feedback. This feedback is stored in system for analysis. This step ensures that user input is not only recorded but also utilized to improve generation of user stories and enhance the rules that guide story creation.

3.2.5 Module 5: Version Control and Tracking Module

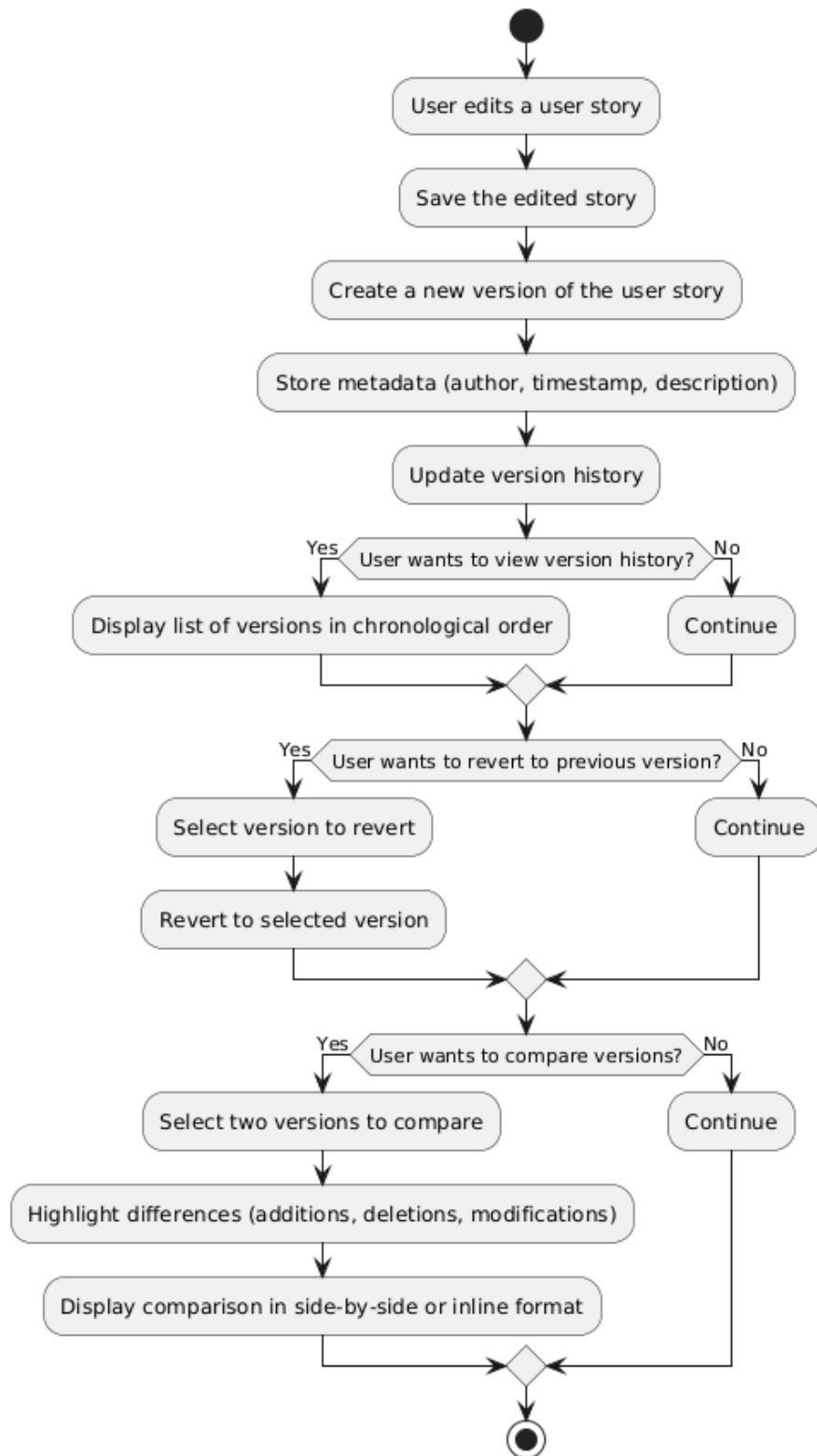


Figure 3.11: Module 5: Activity Diagram

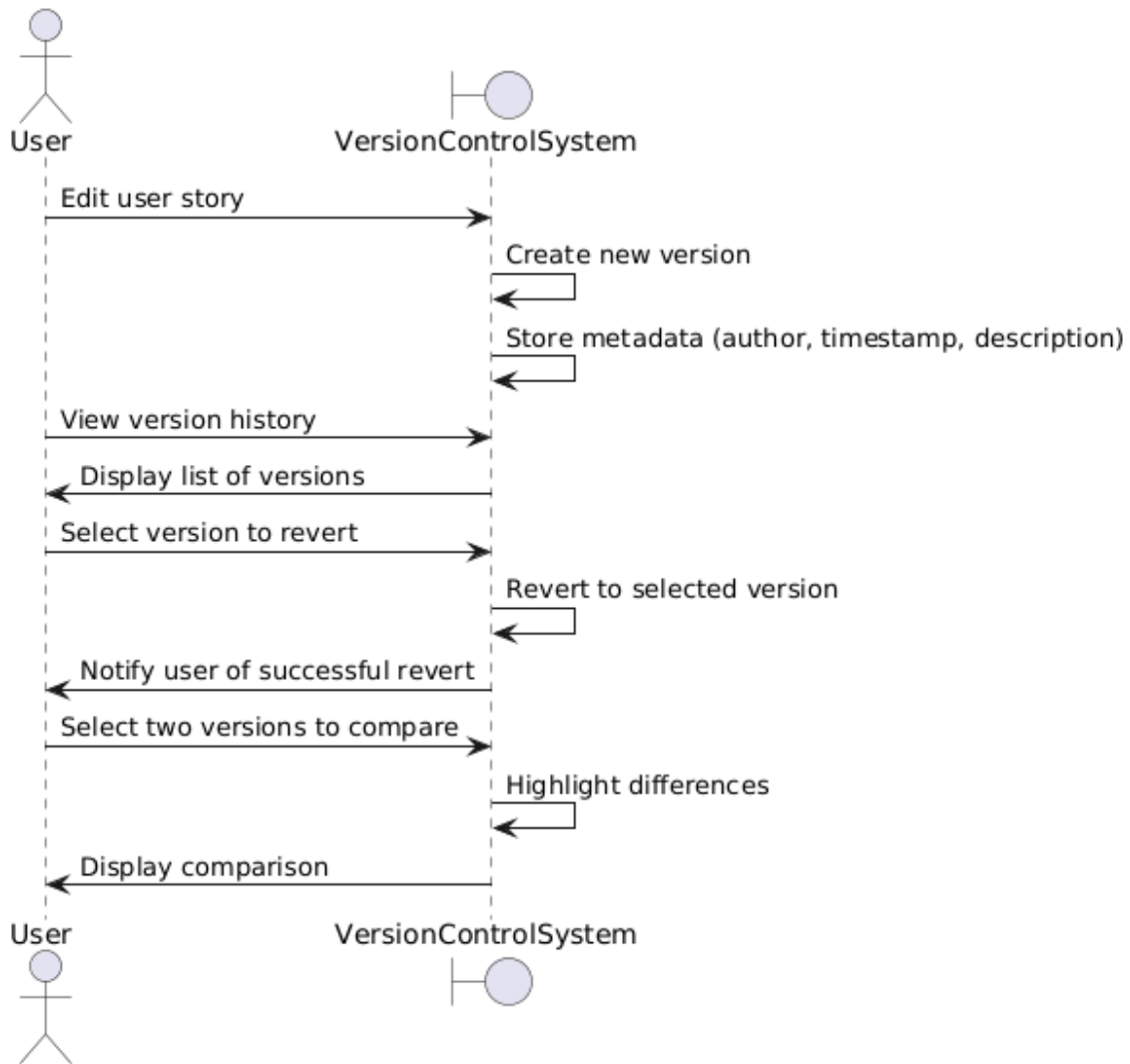


Figure 3.12: Module 5: System Sequence Diagram

Figure 3.11 and Figure 3.12 outline the process for managing multiple versions of user story. After user performs modification and save edits, it prompts system to create new version of the user story. Along with the new version, the system also stores other important information such as the author who performed changes, the timestamp, and a description of the modifications made. This helps in organizing and maintaining the version history for future reference.

Once version is created, system automatically updates history. User has option to view history and if he wishes to view it, system will display it in chronological timeline allowing you to browse through past changes.

If the user wishes to revert to a previous version, they can select the version they want to restore. The system then reverts the user story to the selected version, effectively undoing the more recent changes.

This module also allow user to compare different version of user story. User must select 2 versions from the list. Comparison is displayed side by side. This ensures efficient version control, allowing users to track, compare and undo changes seamlessly.

3.2.6 Module 6: Security and Authentication Module

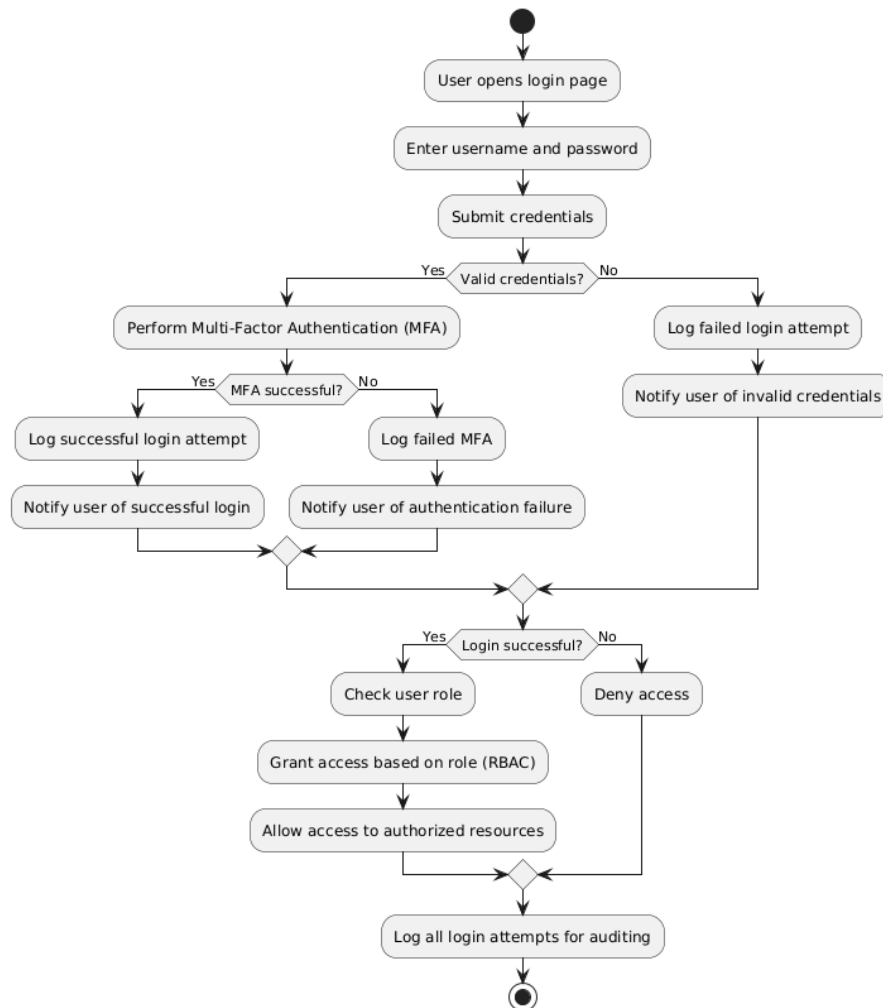


Figure 3.13: Module 6: Activity Diagram

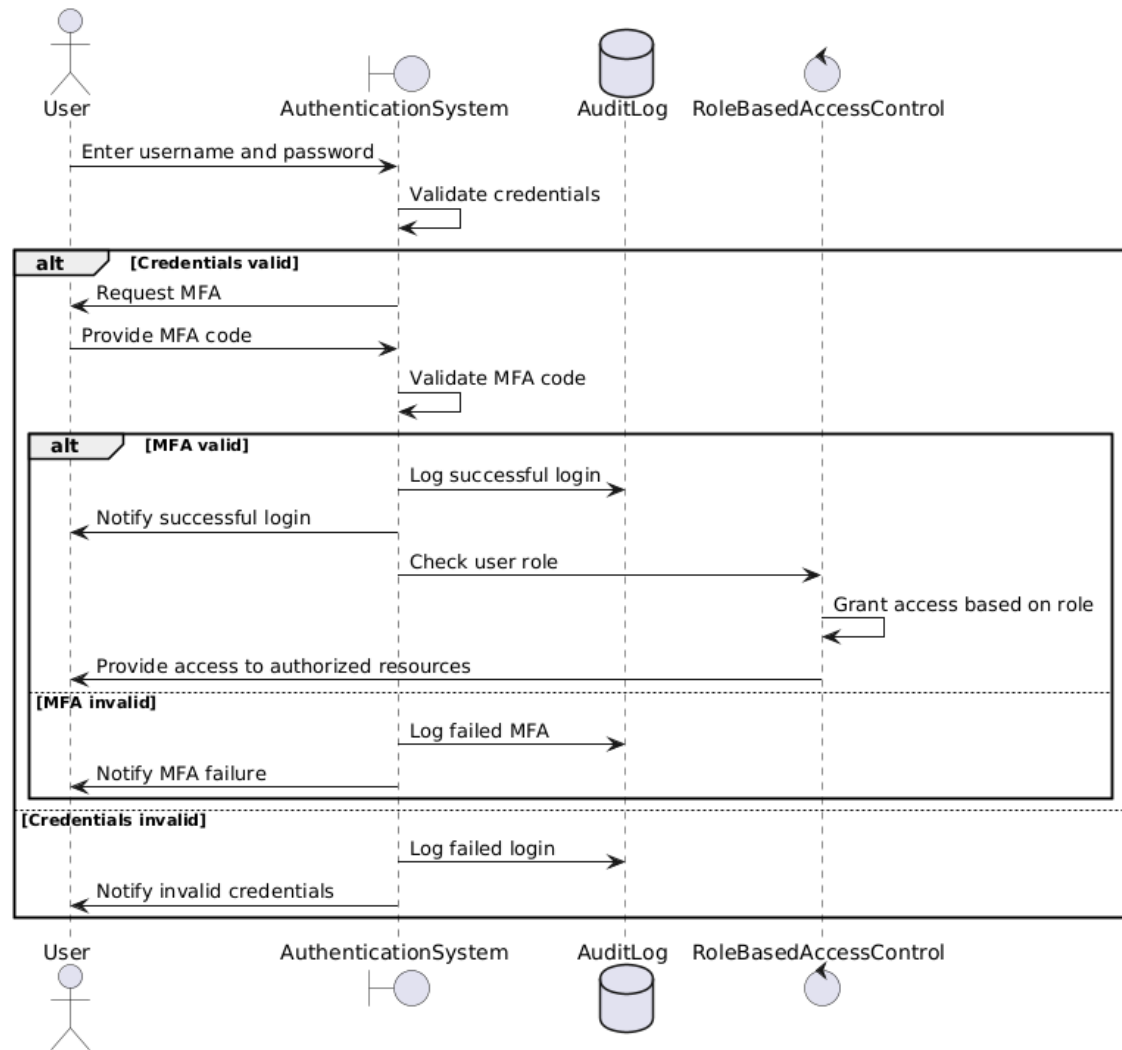


Figure 3.14: Module 6: System Sequence Diagram

Figure 3.13 and Figure 3.14 represent flow of this module. It handles user login procedure and role-based access control (RBAC). When user opens login page of application, he/she is asked to enter their credentials. Once submitted, the system check if credential are valid by cross checking it with database. If invalid, user is asked to enter id and password again and makes sure that user is unable to access any sensitive information.

If credentials are valid, system proceeds to perform Multi Factor Authentication(MFA). Outcome of this step will decide if user should be allowed to access information or not. Authentication code is sent via email to user and system will be waiting for user to enter authentication code. If code entered is invalid, then he will not be allowed to access data and if code is valid then user will be allowed to access application.

Upon successful login, system check user roles and grants access to resources based on their role through RBAC. This will make sure that any user can not access data that is not

meant for him. The system then logs all login attempts for auditing purposes, creating a complete record of both successful and unsuccessful login attempts which will allow system to check for pattern that might indicate if system is being hacked or not.

The overall design aims to enhance security by integrating multiple layers of validation, auditing, and role-based access, ensuring proper control over resources that are being accessed by user.

3.3 Data Design

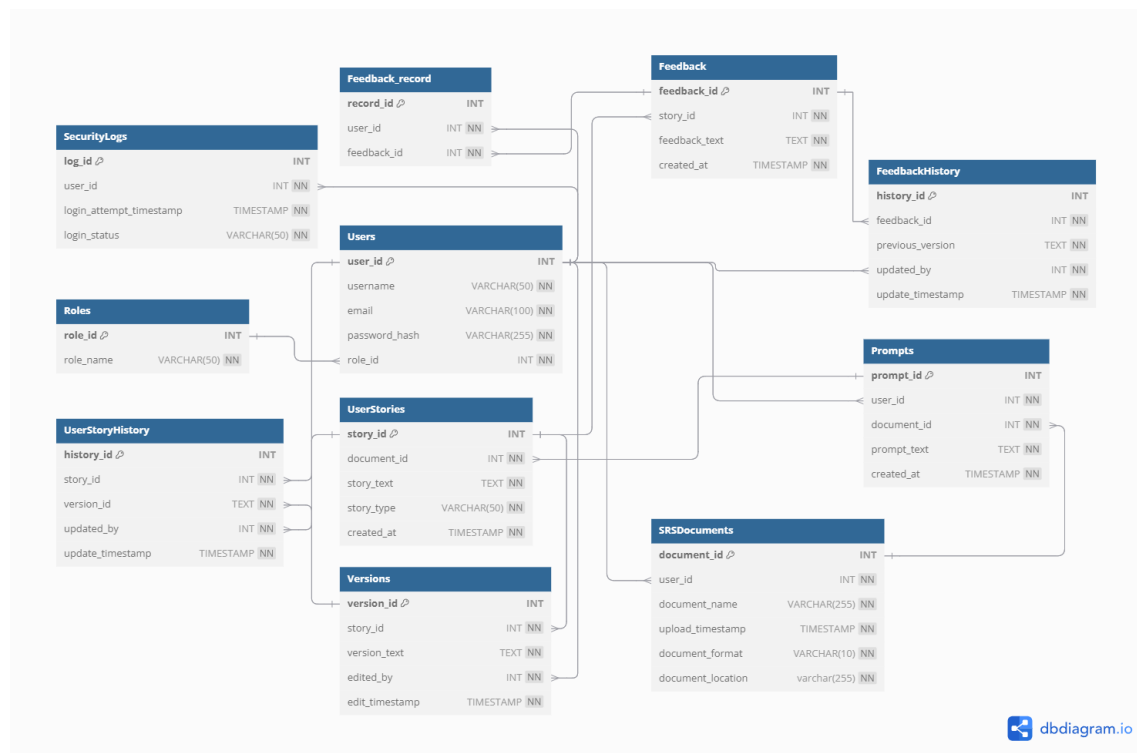


Figure 3.15: Schema Design for complete architecture